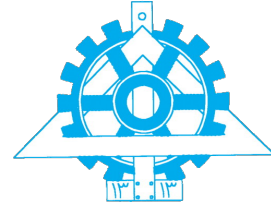




**University of Tehran
College of Engineering
School of Electrical and
Computer Engineering**



Machine Learning

Final Project Phase 2

Group Members

Artin Tavasoli, Parsa Bukani, Shahab Sherafat

February 16, 2026

Contents

1	Data Cleaning and Feature Extraction	1
1.1	Dataset Structure	1
1.2	Audio Standardization	1
1.3	Feature Extraction	2
1.4	Final Feature Representation	3
1.5	Discussion	4
2	Classification	5
2.1	Problem Formulation	5
2.2	Preprocessing Pipeline	5
2.3	Model Selection and Hyperparameter Optimization	6
2.4	Evaluation Metrics	6
2.5	Results	7
2.6	Confusion Matrix Analysis	7
2.7	Final Model Selection	10
3	Clustering	11
3.1	Data Loading and Preprocessing	11
3.2	Dimensionality Reduction	11
3.3	Clustering Algorithms	13
3.4	Conclusion	18

List of Figures

1	Confusion matrix for SVM classifier.	8
2	Confusion matrix for MLP classifier.	8
3	Confusion matrix for Decision Tree classifier.	9
4	Confusion matrix for Random Forest classifier.	9
5	Confusion matrix for Gaussian Naïve Bayes classifier.	10
6	PCA projection	12
7	LDA Projection	13
8	Elbow-method to find optimal K	14
9	K-Means Clustering	15
10	K-Distance Graph used to select the epsilon parameter for DB- SCAN	16
11	DBSCAN Clustering	17

Data Cleaning and Feature Extraction

In this section, the raw multilingual speech dataset collected in Phase 1 is prepared for machine learning tasks. The dataset consists of 712 audio clips (approximately one minute each) in four languages: German, Italian, Korean, and Spanish. The objective of this stage is to ensure consistent audio preprocessing and to transform raw waveforms into discriminative numerical feature representations suitable for classification and clustering.

1.1 Dataset Structure

The dataset is organized hierarchically by language and gender:

- German (Male, Female)
- Italian (Male, Female)
- Korean (Male, Female)
- Spanish (Male, Female)

All files are in MP3 format and originate from audiobook recordings. Since the recordings come from different narrators and production conditions, preprocessing is necessary to standardize the signals before feature extraction.

1.2 Audio Standardization

To ensure consistency across all samples, the following preprocessing steps were applied:

1. **Resampling:** All audio files were resampled to 16 kHz. This sampling rate is standard in speech processing and sufficiently captures the frequency range relevant to human speech while reducing computational cost.
2. **Mono Conversion:** All recordings were converted to mono. Stereo information does not provide additional linguistic content for language identification and may introduce unnecessary variability.
3. **Silence Trimming:** Leading and trailing silence were removed using an energy-based threshold. This step ensures that extracted features focus on speech content rather than empty segments.
4. **Amplitude Normalization:** Each audio signal was normalized to the range $[-1, 1]$ by dividing by its maximum absolute amplitude. This prevents differences in recording volume from biasing the learning algorithms.

These preprocessing steps ensure that all audio samples share the same temporal resolution, amplitude scale, and structural consistency.

1.3 Feature Extraction

Raw waveforms cannot be directly used in traditional machine learning algorithms. Therefore, each cleaned audio signal was transformed into a fixed-length numerical feature vector.

The following features were extracted:

- **Mel-Frequency Cepstral Coefficients (MFCCs):** 13 coefficients were computed to capture perceptually relevant spectral characteristics of speech. MFCCs model the shape of the vocal tract and are widely used in speech recognition tasks.

- **Chroma Features:** These represent the distribution of spectral energy across pitch classes and provide information about harmonic content.
- **Spectral Centroid:** Indicates the center of mass of the spectrum and reflects the brightness of the sound.
- **Spectral Bandwidth:** Measures the spread of the spectrum around its centroid.
- **Zero-Crossing Rate (ZCR):** Captures the rate at which the signal changes sign and provides information about signal noisiness and temporal structure.

Since these features are computed frame-by-frame, temporal statistics were applied to obtain fixed-length representations. For each feature type, both the mean and standard deviation across time were calculated. This approach preserves overall distributional characteristics while ensuring that each audio clip is represented by a consistent feature vector regardless of duration.

1.4 Final Feature Representation

After processing all 712 audio files, each clip was represented by a single fixed-dimensional feature vector. The resulting dataset can be described as:

$$X \in \mathbb{R}^{N \times d}$$

where $N = 712$ is the number of audio samples and d is the dimensionality of the extracted feature vector.

The corresponding label vector:

$$y \in \{\text{German, Italian, Korean, Spanish}\}$$

contains the ground-truth language for each sample.

1.5 Discussion

The chosen preprocessing and feature extraction pipeline ensures:

- Consistent sampling rate and amplitude scale
- Removal of irrelevant silence
- Robust representation of spectral and temporal speech characteristics
- Fixed-length numerical vectors suitable for classical machine learning models

The selected features are well-established in speech processing literature and are expected to provide sufficient discriminative power for distinguishing between languages in both supervised (classification) and unsupervised (clustering) settings.

This processed feature matrix serves as the input to the subsequent Classification and Clustering stages of the project.

Classification

2.1 Problem Formulation

The task of language identification was formulated as a supervised multi-class classification problem with four target languages: German, Italian, Korean, and Spanish. Each audio sample was represented by a feature vector extracted during the previous stage of the project.

The dataset was divided into training and testing subsets using an 80/20 split with stratified sampling. Stratification ensures that the class distribution remains consistent across both subsets, preventing biased evaluation due to class imbalance. The test set was strictly held out and used only once for final performance assessment.

2.2 Preprocessing Pipeline

To ensure stable and fair model comparison, all classifiers were implemented using a unified preprocessing pipeline.

Feature Standardization All features were normalized using `StandardScaler`, transforming them to zero mean and unit variance. This step is particularly important for distance-based and gradient-based models such as SVM and MLP, which are sensitive to feature scale.

Dimensionality Reduction via LDA We applied Linear Discriminant Analysis (LDA) with three components. Unlike PCA, which maximizes overall variance, LDA explicitly maximizes class separability. Since the problem contains four classes, LDA can produce at most $C - 1 = 3$ discriminative com-

ponents. This transformation reduces dimensionality while preserving class-discriminative structure, improving computational efficiency and potentially enhancing generalization.

Embedding preprocessing inside a Pipeline ensures that transformations are applied consistently during cross-validation and testing, preventing data leakage.

2.3 Model Selection and Hyperparameter Optimization

Five classification models were evaluated:

- Support Vector Machine (RBF kernel)
- Multi-Layer Perceptron (Neural Network)
- Decision Tree
- Random Forest
- Gaussian Naïve Bayes

Hyperparameters for SVM, MLP, Decision Tree, and Random Forest were optimized using GridSearchCV with 5-fold stratified cross-validation. The primary optimization metric was the macro-averaged F1-score.

Using macro F1-score ensures that all language classes are treated equally during model selection, avoiding bias toward more easily separable or more frequent classes.

2.4 Evaluation Metrics

Final evaluation was performed on the held-out test set using:

- Accuracy
- Precision (Macro and Weighted)

- Recall (Macro and Weighted)
- F1-score (Macro and Weighted)
- Confusion Matrix

Accuracy provides overall correctness, while macro-averaged metrics treat all classes equally. Weighted metrics incorporate class frequency. Reporting both provides a balanced and comprehensive evaluation of performance.

2.5 Results

Experimental results demonstrate extremely high classification performance across most models. SVM (RBF), MLP, Random Forest, and Gaussian Naïve Bayes achieved perfect or near-perfect performance on the test set.

The Decision Tree model exhibited minor misclassifications. Inspection of its confusion matrix shows limited confusion between Korean and Spanish samples, suggesting that shallow tree-based partitioning may struggle with certain fine-grained acoustic similarities.

The near-perfect performance of multiple models indicates that the extracted acoustic features, combined with LDA-based dimensionality reduction, produce highly separable representations of the four languages.

2.6 Confusion Matrix Analysis

Figures 1–5 illustrate the confusion matrices for all evaluated classifiers.

The confusion matrices confirm that most models achieve perfect diagonal dominance, meaning that samples are correctly classified across all language categories. Only the Decision Tree model exhibits minimal off-diagonal entries, indicating slight boundary instability.

This analysis reinforces the conclusion that the feature representation is highly discriminative and that margin-based (SVM) and ensemble-based (Random Forest) approaches provide strong generalization performance.

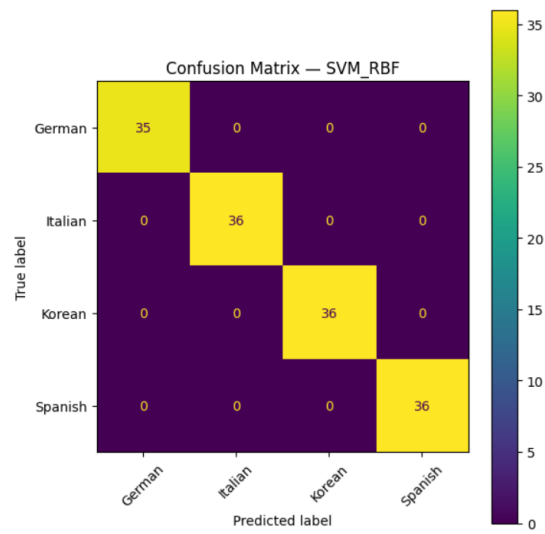


Figure 1: Confusion matrix for SVM classifier.

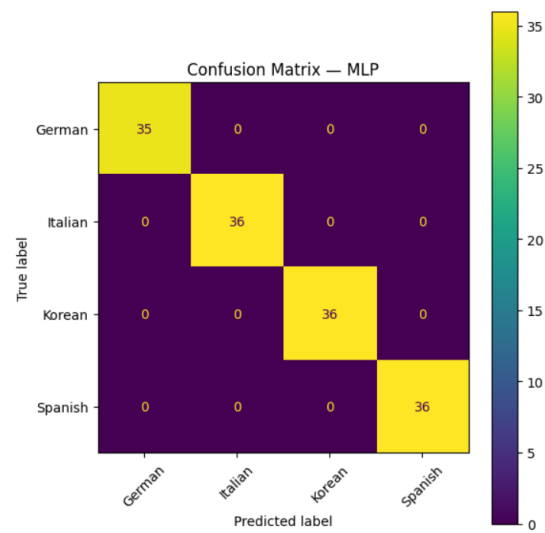


Figure 2: Confusion matrix for MLP classifier.

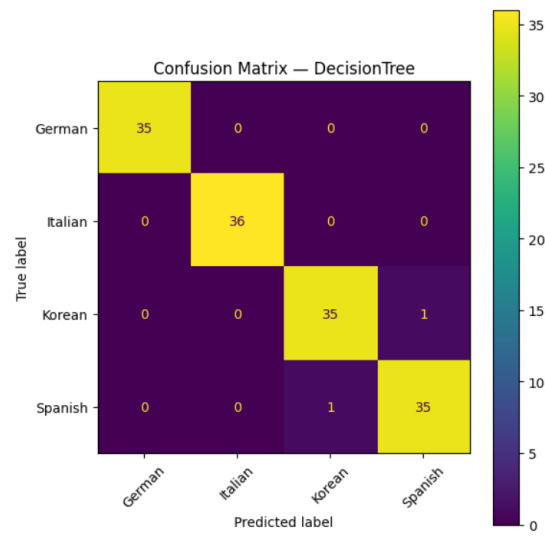


Figure 3: Confusion matrix for Decision Tree classifier.

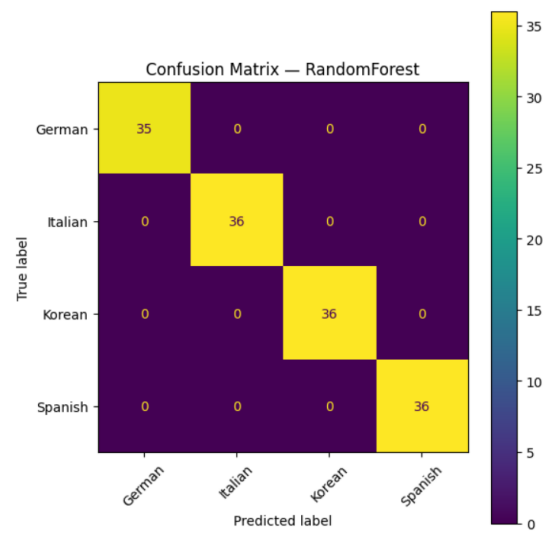


Figure 4: Confusion matrix for Random Forest classifier.

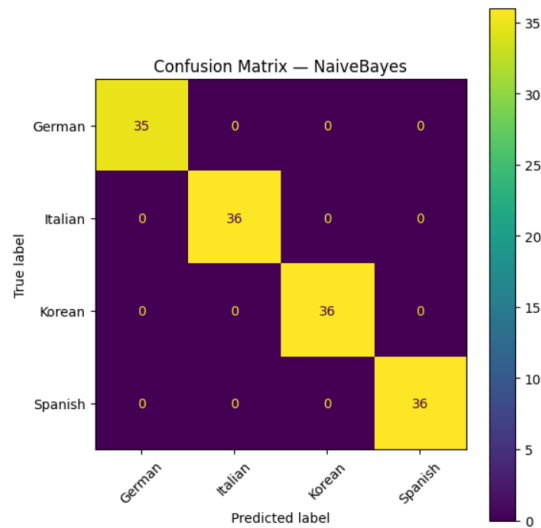


Figure 5: Confusion matrix for Gaussian Naïve Bayes classifier.

2.7 Final Model Selection

Since multiple models achieved identical macro F1-scores, model selection was guided by stability and theoretical generalization properties.

Support Vector Machines offer strong margin maximization properties, leading to robust decision boundaries. Random Forest provides ensemble stability and reduced variance compared to single trees.

Considering both theoretical robustness and empirical performance, SVM (RBF kernel) and Random Forest are selected as the most reliable classifiers for this task.

Clustering

We want to classify audio samples into four distinct languages (German, Italian, Korean, and Spanish) using unsupervised machine learning techniques. The dataset comprises 712 data points, with each point represented by 56 extracted features.

3.1 Data Loading and Preprocessing

We began by loading the features matrix (X) and the corresponding labels (y).

- **Features shape:** (712, 56)
- **Labels shape:** (712,)

Standardization: Before applying any distance-based algorithms, it is critical to scale the data. Raw audio features often have varying magnitudes, which can bias algorithms like K-Means towards features with larger numerical values. We applied a Standard Scaler to the dataset, transforming it such that every feature has a mean of 0 and a standard deviation of 1.

3.2 Dimensionality Reduction

Dealing with 56 dimensions is computationally expensive and difficult to visualize. We explored two methods to reduce the dimensionality of the data to a visualizable space (2D or 3D).

Method A: Principal Component Analysis (PCA) We first applied PCA, an unsupervised linear technique, to reduce the data to 2 dimensions. PCA seeks to find the directions (components) that maximize the variance in the data.

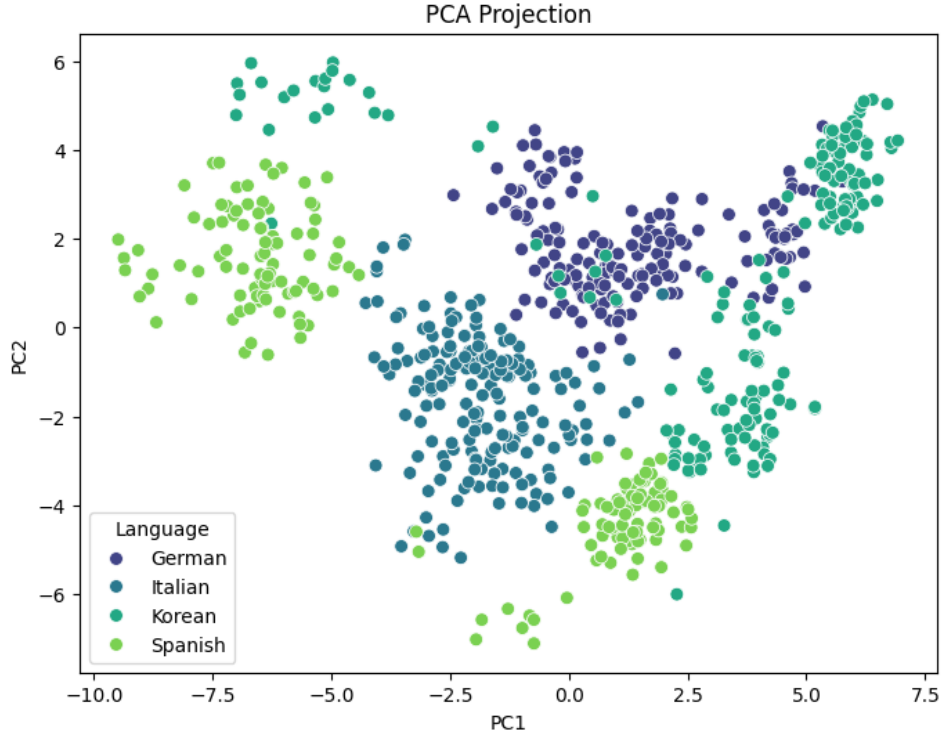


Figure 6: PCA projection

The PCA results revealed that this method was insufficient for our specific dataset. The variance explained by the first two components was only **0.40 (40%)**. As observed in Figure 1, the four language classes overlap significantly. This overlap occurs because PCA preserves global variance without considering class labels, and in this case, the direction of maximum variance did not align with the differences between languages.

Method B: Linear Discriminant Analysis (LDA) To overcome the limitations of PCA, we switched to Linear Discriminant Analysis (LDA). Unlike PCA, LDA is a supervised dimensionality reduction technique. It focuses on maximizing the separability between known classes (maximizing between-class variance while minimizing within-class variance).

Given 4 classes ($C = 4$), LDA is constrained to produce at most $C - 1$ components. We projected the data into **3 dimensions**.

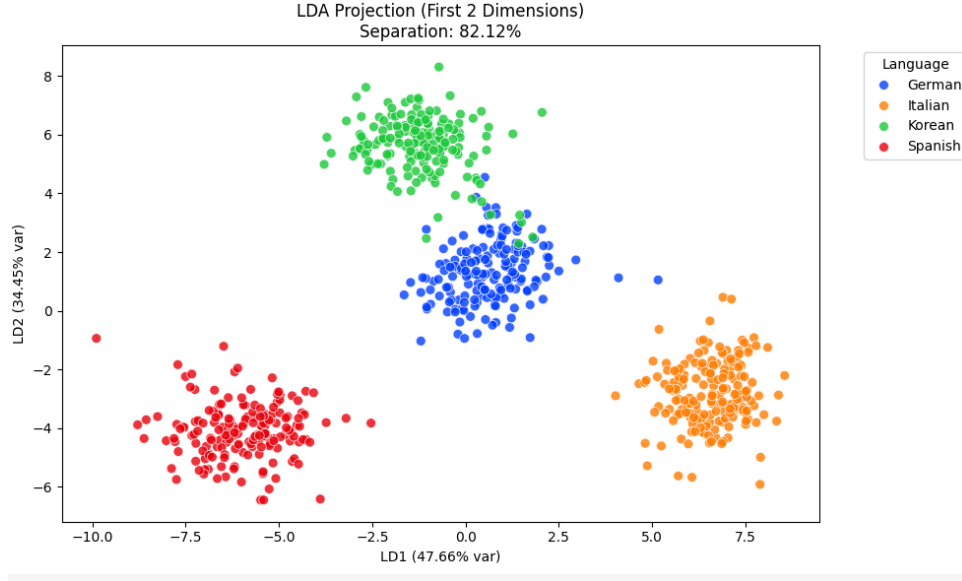


Figure 7: LDA Projection

The transition to LDA provided a drastic improvement. As shown in Figure 2, the four languages formed tight, distinct clusters in the 3D latent space. This confirms that the features possess strong discriminatory power when viewed through a supervised transformation. We used this 3-dimensional LDA space for all subsequent clustering tasks.

3.3 Clustering Algorithms

With the data successfully projected into a separable 3D space, we applied two clustering algorithms to assign group labels without using the ground truth during the clustering process.

Algorithm 1: K-Means Clustering K-Means partitions data into K clusters by minimizing the distance between points and their respective cluster centroids. Finding the Optimal K (Elbow Method) Although we know the ground truth contains 4 languages, it is crucial to verify if the data naturally forms 4 distinct groups in our LDA-transformed feature space. We used the Elbow Method, plotting the Inertia (sum of squared distances to the nearest cluster

center) against different values of K . as expected we see a bend in the curve at $K = 4$ and ran K-Means with $K = 4$ on the 3D LDA features.

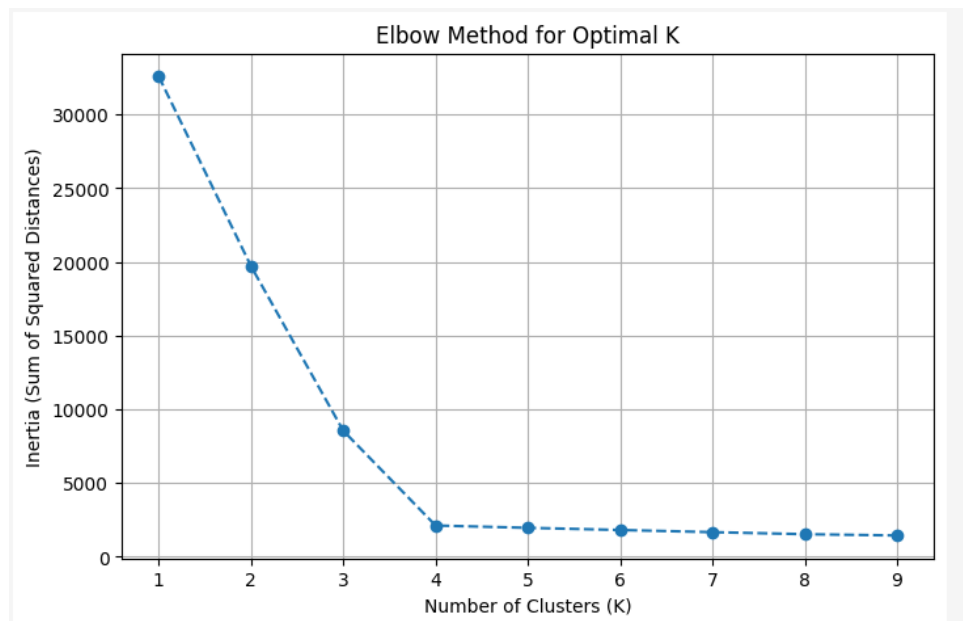


Figure 8: Elbow-method to find optimal K

We trained the K-Means model with $K=4$ on the LDA-transformed data.

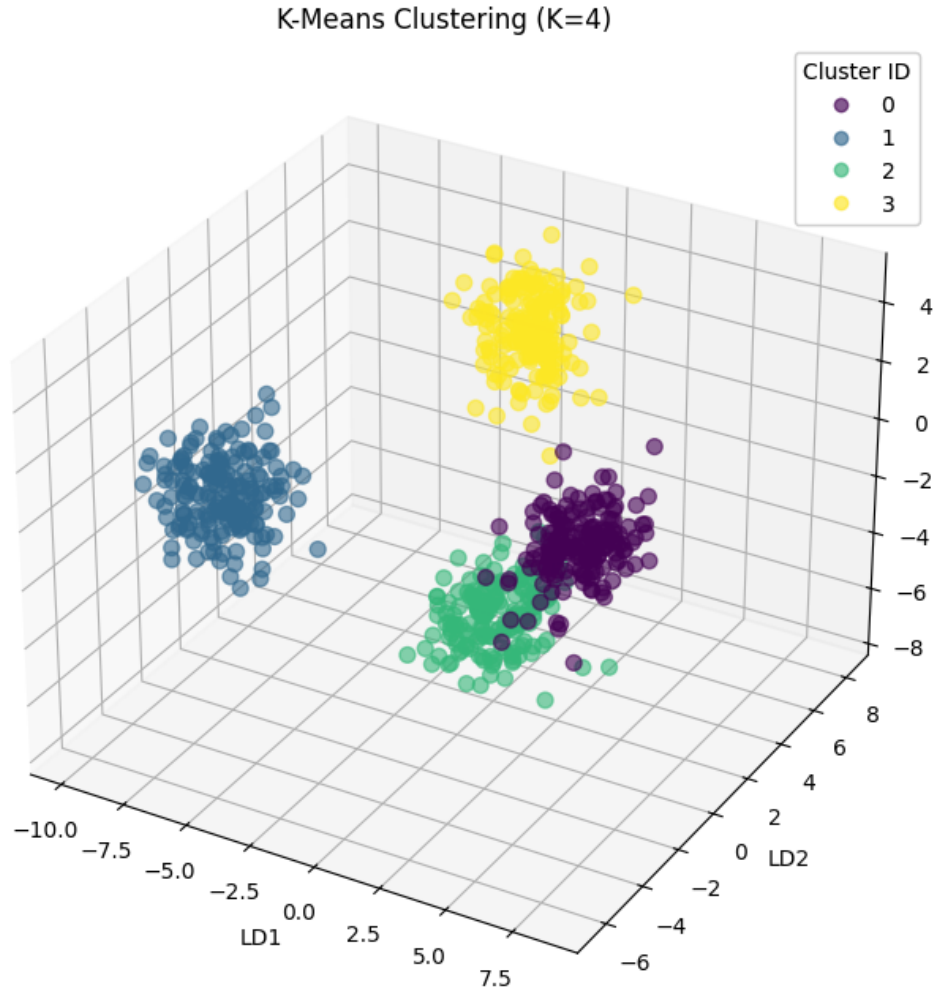


Figure 9: K-Means Clustering

The K-Means algorithm performed perfectly on this dataset (purity 1). Every single data point was correctly assigned to a cluster corresponding to its language. This perfect score is attributable to the LDA transformation, which created convex, isotropic clusters—the exact structure K-Means is designed to detect.

Algorithm 2: DBSCAN We also applied DBSCAN (Density-Based Spatial Clustering of Applications with Noise), which groups points based on density and can identify outliers as noise.

We used a Nearest Neighbors distance graph (k-distance plot) to find the optimal ϵ (epsilon) value. The "knee" of the curve indicates the threshold where

points cease to be neighbors and start becoming outliers.

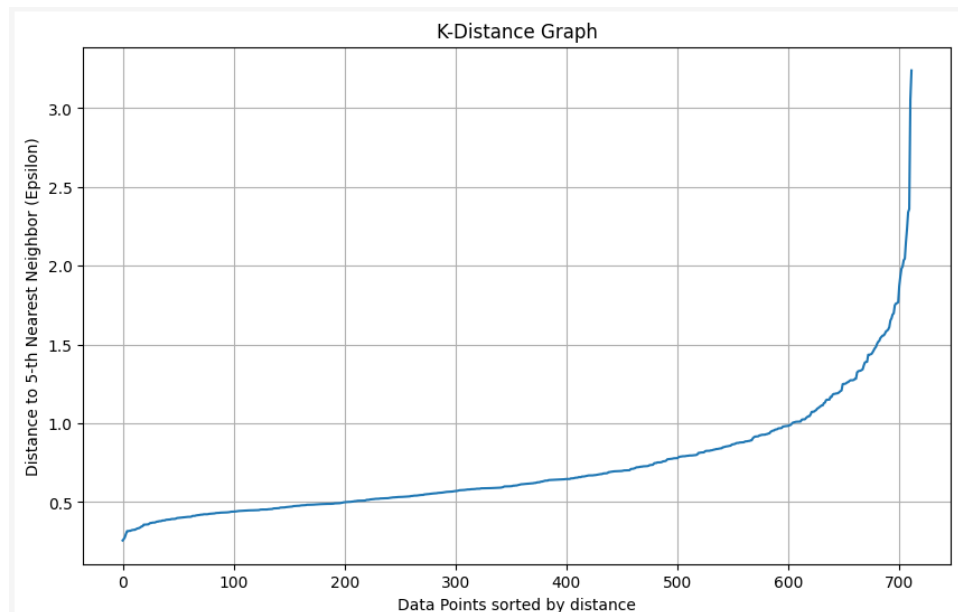


Figure 10: K-Distance Graph used to select the epsilon parameter for DBSCAN

DBSCAN was applied using the selected epsilon. Unlike K-Means, DBSCAN does not require specifying the number of clusters beforehand.

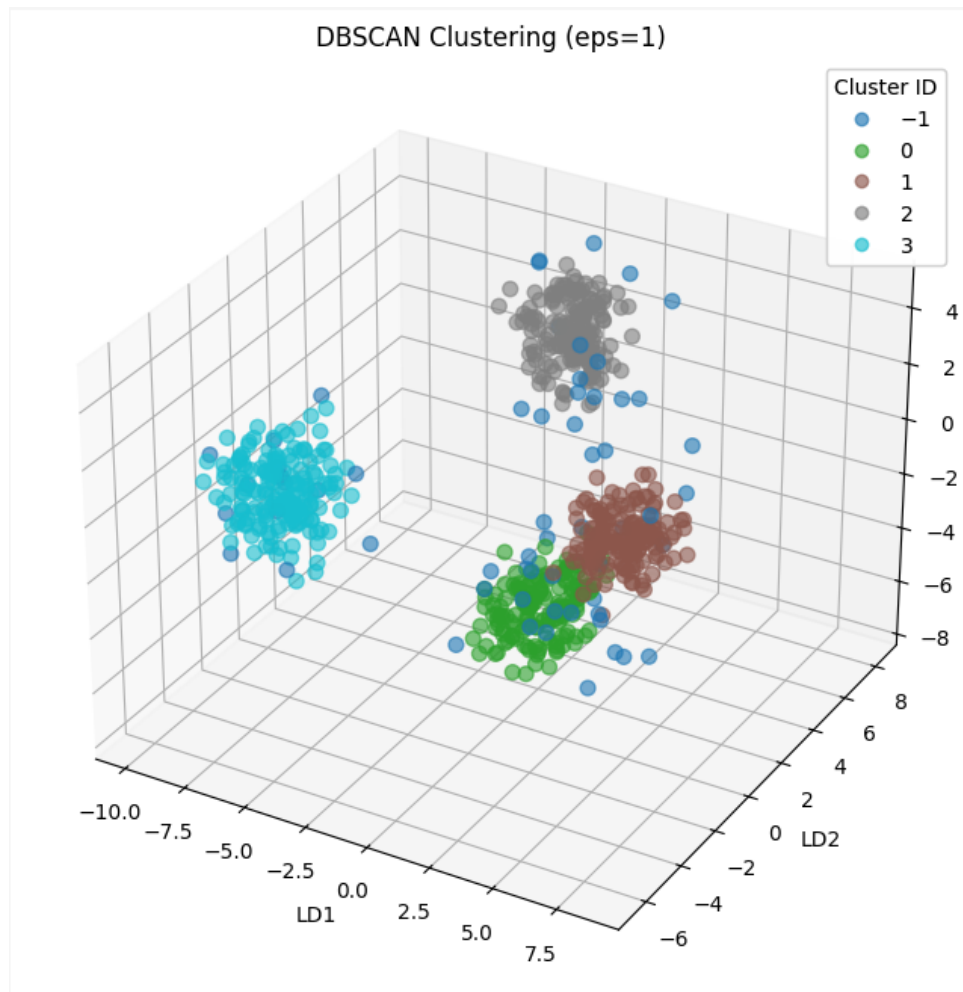


Figure 11: DBSCAN Clustering

Analysis of DBSCAN Performance:

- Overall Purity: 0.9157
- Cluster Composition:
 - Cluster 0: 159 points (100% German)
 - Cluster 1: 164 points (100% Italian)
 - Cluster 2: 163 points (100% Korean)
 - Cluster 3: 166 points (100% Spanish)
 - Noise (Cluster -1): 60 points

DBSCAN successfully identified the four main language groups with high purity. However, it classified approximately 8% of the data as noise (Cluster -1). This occurs because density-based methods are sensitive to the "fuzziness" at the edges of clusters. While LDA separated the centers of the classes well, the outlier points of each language were sparse enough to be dropped by DBSCAN.

3.4 Conclusion

This analysis demonstrated that audio features can effectively distinguish between German, Italian, Korean, and Spanish speakers.

1. Dimensionality Reduction: LDA was significantly superior to PCA, providing a clear 3D separation of classes.
2. Clustering: K-Means achieved perfect classification (1.0 purity) due to the clean separation provided by LDA. DBSCAN also performed well (0.91 purity) but was stricter, treating edge cases as noise. Besides purity, there is another metric which doesn't care about labels; Silhouette Score, It only cares about distances and shapes. It calculates: "How close is this point to its own cluster center?" vs. "How far is it from the next closest cluster?"

K-Means Silhouette Score: 0.7572

DBSCAN Silhouette Score : 0.6525